MINI PROJECT 4

Infrastructure Automation with Terraform and Ansible on AWS

Enterprise Student Record & Academic Management Platform
Unit 4 Theoretical and Architectural Reference

Document Attribute	Specification Details
Student	Gaurav Borle | PRN: 2125UDSM1071
Curriculum Unit	UNIT 4: DevOps Automation (Infrastructure Automation on AWS)
Student Project	Enterprise Student Record & Academic Management Platform
Trainer Reference System	Enterprise Smart Library Management Platform
Cloud Platform	Amazon Web Services (AWS)
Automation Tools	Terraform (infrastructure provisioning) and Ansible (server configuration)
Source Repository	https://github.com/GauravBorle04/Devops-Miniprojects-SY
Document Purpose	Theory and design reference explaining why and how infrastructure is automated; implementation is built on this design

1. Introduction: The Need for Infrastructure Automation

In the earlier units the Student Record platform was designed, coded, tested, containerized with Docker and run on Kubernetes. All of that work, however, ran on a server that was created and prepared by hand: the instance was launched by clicking through the AWS console and the tools were installed by typing commands over SSH. That is acceptable once, but it does not scale and it cannot be repeated reliably.

Before an application can serve users, an organisation must build a whole supporting environment: a private network, subnets, routing, firewall rules, a compute instance, storage, operating-system packages, a container runtime and a web server. Doing this manually (often called "ClickOps") causes slow provisioning, undocumented changes, security mistakes and servers that slowly drift away from each other. Unit 4 solves this with two complementary ideas: **Infrastructure as Code (IaC)**, where the cloud environment is described in text files, and **Configuration Management**, where the software on each server is set up by repeatable, automated playbooks.

This document explains the theory behind **Terraform** (provisioning the AWS infrastructure) and **Ansible** (configuring the server and deploying the application), and shows how they combine into one automated flow for the Student Record platform.

> **CORE OBJECTIVE**
> Replace manual server setup with declarative Infrastructure as Code (Terraform) and idempotent configuration management (Ansible).
>
> The result is an environment that can be rebuilt on demand, is identical every time, is recorded in Git, and can be reviewed like any other code.

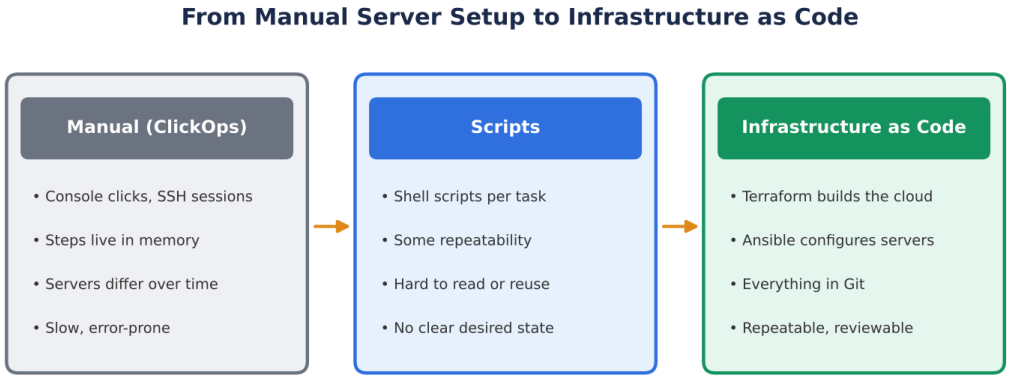


Figure: Evolution of infrastructure management from manual setup to Infrastructure as Code

2. Infrastructure Requirements Analysis

Infrastructure planning always comes before provisioning. The first step is to list what each part of the application needs and map it to an AWS building block. This avoids over-paying for capacity that is not needed and avoids leaving security gaps.

2.1 Application Components Mapped to AWS Services

Component	Function	AWS / Platform Service	Specification
Python web API	Flask REST API and dashboard served by a WSGI server	EC2 instance (compute)	Ubuntu LTS, small instance (t2.micro in the reference design), 20 to 30 GB gp3 root volume
Data store	Students, courses, enrollments, marks	In-memory / local store; exportable	Thread-safe store inside the container; a managed database such as RDS is the upgrade path
Reverse proxy	Accepts web traffic and forwards it to the app	Nginx on the instance	Listens on port 80, forwards to the container on port 5000
Container runtime	Runs the packaged application	Docker Engine	Container restart policy "always"; port mapping 5000:5000
Virtual network	Private, isolated network boundary	VPC with public subnet	VPC 10.1.0.0/16, public subnet 10.1.1.0/24, Internet Gateway and route table
Network security	Filters inbound and outbound traffic	Security Group (stateful)	Inbound 22 (SSH), 80 (HTTP) and 8080 (Jenkins); outbound open
Backup and state	Backups and Terraform state	S3 bucket (+ DynamoDB lock)	Private, encrypted, versioned bucket; DynamoDB table for state locking

2.2 Environment Variations

- **Development:** minimal cost and fast feedback: one small instance in a single zone, relaxed rules for testing.
- **Staging:** a faithful copy of production, used for integration tests, load tests and security scans.
- **Production:** high availability, a fixed public address, encrypted backups, least-privilege access and alerting.

A good automation design supports all three from the same code, changing only a few variables.

3. Cloud Architecture Design on AWS

A cloud design separates the part that is open to the internet from the part that runs the application. The platform lives inside its own private network, and only the ports it needs are opened.

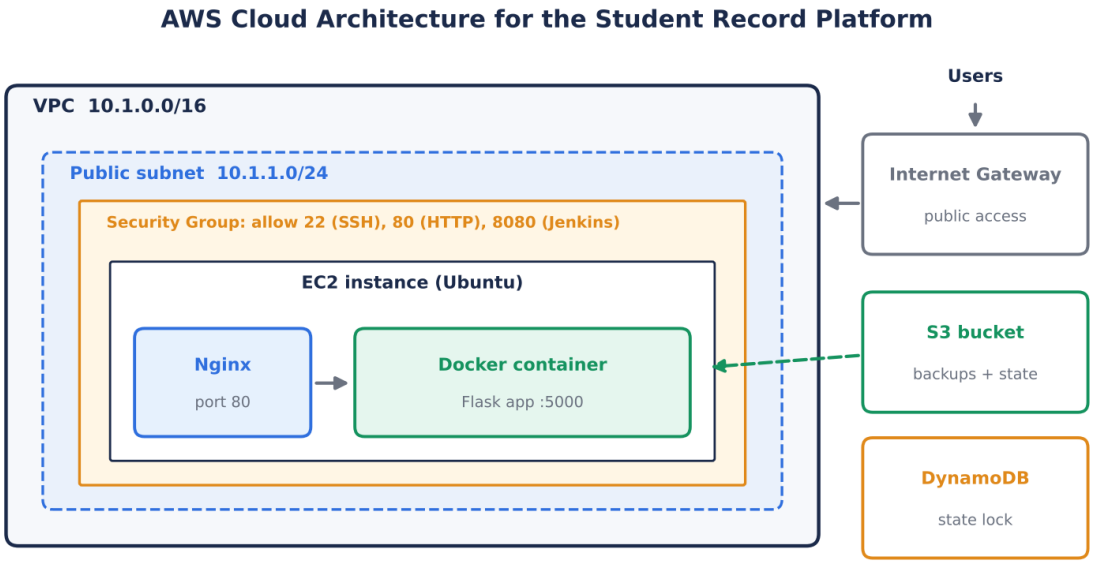


Figure: AWS cloud architecture for the Student Record platform

3.1 Architectural Layers

- **Network isolation:** a custom Virtual Private Cloud (VPC) with the address range 10.1.0.0/16. An Internet Gateway connects it to the internet, and a public subnet (10.1.1.0/24) holds the server, with a route sending all outside traffic (0.0.0.0/0) to the gateway.
- **Security:** a stateful Security Group acts as a firewall. Because it is stateful, replies to allowed requests are allowed automatically. Only SSH (22), HTTP (80) and the Jenkins port (8080, carried over from Unit 3) are open inbound.
- **Compute and runtime:** an EC2 instance running Ubuntu with Docker. The Student Record container listens on port 5000, and Nginx on port 80 forwards visitors to it.
- **State and backup:** an S3 bucket stores versioned, encrypted backups and the Terraform state file, while a DynamoDB table provides a lock so two people cannot change infrastructure at the same moment.

4. Infrastructure as Code with Terraform

Infrastructure as Code means managing infrastructure through machine-readable definition files instead of clicking in a web console. **Terraform**, from HashiCorp, is the most widely used tool for this. You write what should exist; Terraform works out what has to be created, changed or removed to make reality match.

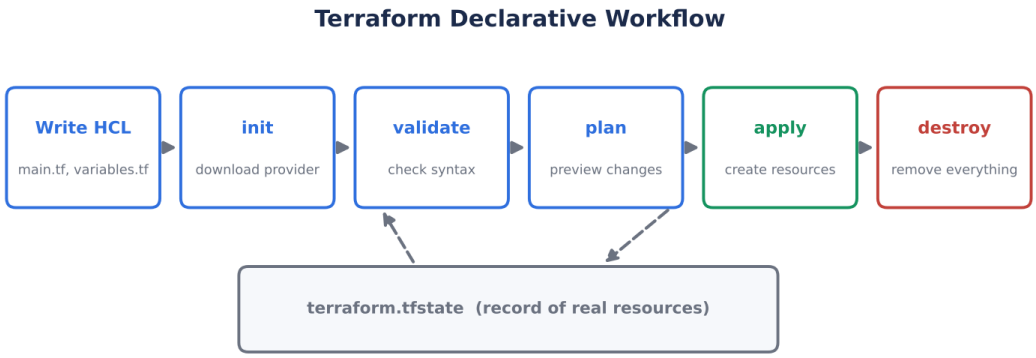


Figure: Terraform declarative workflow and state file

4.1 Core Concepts

- **Declarative language (HCL):** you describe the desired end state, for example "one EC2 instance in this subnet", not the sequence of steps.
- **Providers:** plugins that talk to a platform's API. The AWS provider turns your declarations into AWS API calls.
- **Resources and data sources:** a resource is something Terraform creates (a VPC, an instance); a data source reads something that already exists (for example the latest Ubuntu image).
- **Variables and outputs:** variables make the code reusable (instance type, region); outputs return values after creation, such as the server's public IP.
- **State (terraform.tfstate):** a record linking each declared resource to the real object in AWS. It is the single source of truth, so it must be protected and never edited by hand.
- **State locking:** in a team, two simultaneous runs could corrupt the state. A remote backend (S3 with a DynamoDB lock) prevents this.

4.2 The Terraform Command Lifecycle

Command	What it does	Outcome
terraform init	Prepares the working folder, downloads the AWS provider and connects the backend	Creates the .terraform folder and a dependency lock file
terraform validate	Checks the files for syntax and structure errors without calling AWS	Confirms the configuration is well formed
terraform plan	Compares the code with the real infrastructure and prepares a preview	Lists resources to be created (+), changed (~) or destroyed (-)
terraform apply	Carries out the plan, creating resources in dependency order	Builds the VPC, subnet, security group, S3 and EC2, then updates the state
terraform destroy	Removes everything Terraform manages, in reverse order	Deletes the resources and stops the cost

The **plan** step is the safety net: you can read exactly what will happen before anything is changed. Terraform also builds a dependency graph automatically; for example, the instance needs the subnet, so the subnet is created first, and independent resources are created in parallel.

4.3 Planned Terraform Resources for This Project

Resource	Purpose
aws_vpc	Private network 10.1.0.0/16

Resource	Purpose
aws_subnet	Public subnet 10.1.1.0/24
aws_internet_gateway and aws_route_table	Connect the subnet to the internet
aws_security_group	Allow ports 22, 80 and 8080 inbound
aws_key_pair	Public key used for SSH access
aws_instance	The EC2 server that will run the application
aws_s3_bucket	Versioned, encrypted bucket for backups
output	Publishes the server's public IP for Ansible

An example of how a resource is declared (illustrative):

```
resource "aws_instance" "app_server" {
  ami                    = data.aws_ami.ubuntu.id
  instance_type          = var.instance_type
  subnet_id              = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.app_sg.id]
  key_name               = aws_key_pair.deployer.key_name
  tags = { Name = "student-platform-server" }
}

output "app_server_public_ip" {
  value = aws_instance.app_server.public_ip
}
```

> **SECURITY NOTE**
> Never write AWS access keys inside Terraform files or commit them to Git. The AWS CLI stores credentials in a local file that Terraform reads automatically.
>
> Likewise the SSH private key (.pem) and any tokens must stay out of the repository.

5. Configuration Management with Ansible

Terraform builds the cloud "hardware", but it is not designed to install software or tune an operating system. That job belongs to a configuration management tool, and **Ansible** is the one used here.

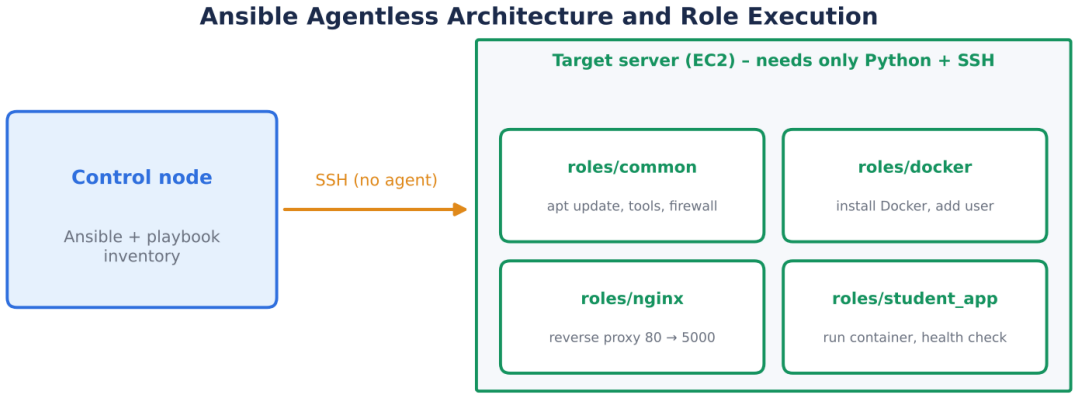


Figure: Ansible agentless control node and role execution

5.1 Why Ansible

- **Agentless:** Ansible connects over ordinary SSH. The target server only needs Python, so nothing extra has to be installed or kept running on it.
- **Push-based:** the control node pushes the configuration when you run it, with no central server or waiting for clients to check in.
- **Idempotent:** running a playbook twice gives the same result as once. If Docker is already installed, Ansible reports "ok" and skips it, so a playbook is safe to re-run at any time.
- **Readable YAML:** playbooks are plain text that is easy to review and share.

5.2 Core Building Blocks

Term	Meaning
Inventory	The list of servers Ansible manages and how to reach them
Playbook	A YAML file that says which roles or tasks run on which hosts
Task and module	One action, performed by a module such as apt, service or docker_container
Role	A reusable folder of tasks, templates and defaults for one concern
Handler	A task that runs only when something changed, such as restarting Nginx
Variables (group_vars)	Values that differ per environment while tasks stay identical

5.3 Role Structure for the Student Record Platform

- **roles/common:** refreshes package lists, installs basic tools (curl, git, jq) and sets up the firewall.
- **roles/docker:** adds Docker's official repository, installs Docker and its runtime, enables the service and lets the login user run Docker.
- **roles/nginx:** installs Nginx and uses a template to forward port 80 to the container on port 5000.
- **roles/student_app:** runs the Student Record container from Unit 3 with a restart policy and checks that the health endpoint answers.

A top-level playbook then simply lists the roles in order:

```
- hosts: student_servers
  become: true
  roles:
    - common
    - docker
    - nginx
    - student_app
```

6. The Two-Tier Orchestration Handshake

Terraform and Ansible do different jobs, so they are used one after the other: Terraform first, then Ansible. The challenge is that Terraform only learns the server's public IP after it has been created, while Ansible needs that address in its inventory.

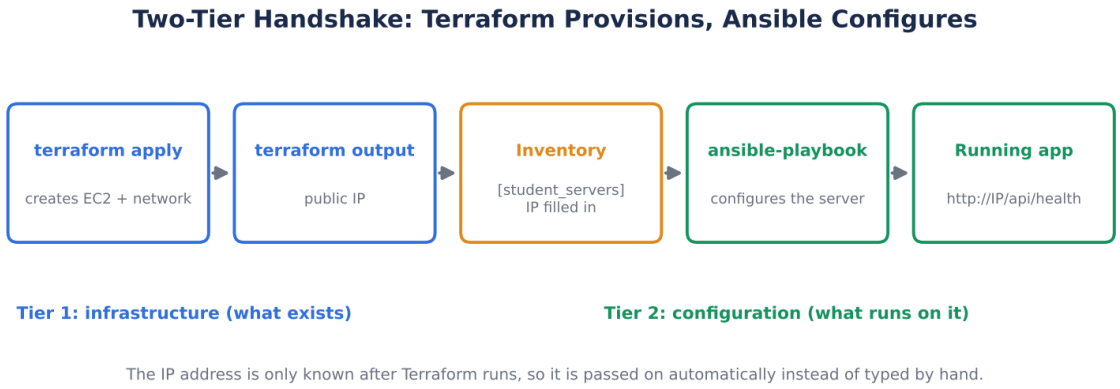


Figure: Terraform provisions, Ansible configures

6.1 The Handoff

```
# 1. Terraform creates the server and prints its public IP
terraform apply

# 2. A script reads the output
terraform output -raw app_server_public_ip

# 3. The IP is written into the Ansible inventory
[student_servers]
student-host ansible_host=<PUBLIC_IP> ansible_user=ubuntu

# 4. Ansible configures the new server
ansible-playbook -i inventory playbook.yml
```

This hand-off can be done by a small shell script or by a CI pipeline. Because Terraform outputs and the Ansible inventory are connected automatically, the whole environment can be created from nothing with no manual copying of values.

7. Environment Parity and Promotion

Many deployment failures come from environments that are quietly different: another operating-system patch level, a different Python version or a changed firewall. Software that passed its tests then fails in production. Infrastructure as Code removes this problem because every environment is created from the same files.

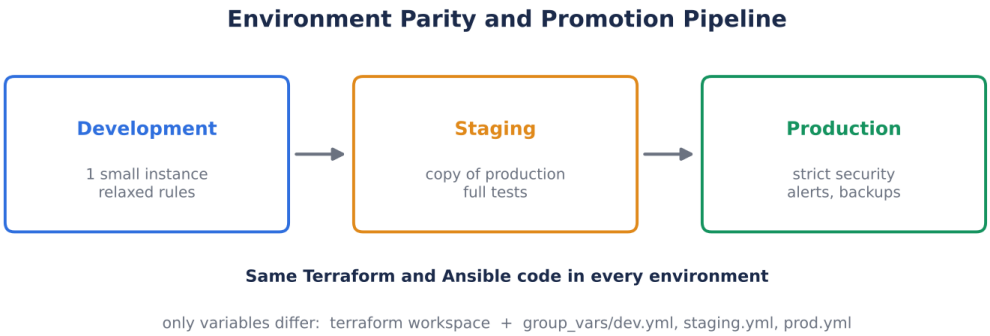


Figure: Environment parity across development, staging and production

7.1 Achieving Parity Through Code

- **Terraform workspaces:** one codebase can create several isolated environments. Switching workspace changes variables such as instance size, not the architecture.
- **Ansible group variables:** environment-specific values live in files like group_vars/dev.yml and group_vars/prod.yml, while the tasks stay exactly the same.
- **Promotion:** a change is proven in development, then staging, then production, always through the same automation.

8. Validation and Security Hardening

Creating infrastructure is not the end: it must be checked. A four-gate validation moves from the network layer up to the application, and a release is accepted only if every gate passes.

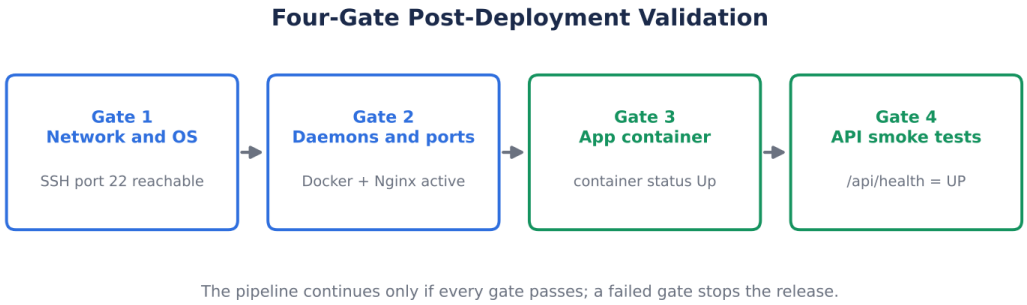


Figure: Four-gate post-deployment validation

Gate	Objective	Example check
1. Network and OS	Server reachable, SSH port open, operating system running	nc -zv -w 5 <IP> 22 and ssh -i key.pem ubuntu@<IP> uptime
2. Daemons and ports	Docker and Nginx services active, port 80 listening	ansible -m systemd -a "name=docker state=started" student_servers
3. Application container	Container status is Up and the port is mapped	docker ps --filter name=student-app
4. API smoke tests	Live HTTP requests to the REST endpoints succeed	curl -s -f http://<IP>/api/health (must show "UP")

8.1 Security Practices

- **Least privilege:** the IAM user used by Terraform should have only the permissions it needs for EC2, VPC and S3.
- **Restricted access:** open SSH only to your own address where possible, and keep keys out of Git.
- **Protect state:** the state file can contain sensitive values, so keep it in a private, encrypted, versioned bucket.
- **Clean up:** run terraform destroy when the work is finished so no resources keep generating charges.

9. Student Project Adaptation

After the trainer's demonstration on the Library platform, the same approach is applied independently to the Student Record platform.

Dimension	Trainer (Library Platform)	Student (Student Record Platform)
Application domain	Book circulation, borrowing, inventory	Student records, course enrollment, academic marks
Compute sizing	EC2 t2.micro (1 vCPU, 1 GB)	EC2 t2.micro in the reference design; use the free-tier-eligible type shown in your AWS account
Terraform network	VPC 10.0.0.0/16; ports 22, 80, 5000	VPC 10.1.0.0/16; ports 22, 80, 8080
Ansible role focus	Docker container running the Library app	Docker container running the Student Record app
API smoke tests	GET /api/health, GET /api/books, POST /api/borrow	GET /api/health, GET /api/students, POST /api/marks

9.1 Evaluation Areas

Deliverable	Weight	What is expected
Terraform configuration files	30%	Clean, modular definitions for VPC, subnet, security group, EC2 and S3 that run without manual intervention
Ansible automation roles	30%	Modular roles for setup, Docker, Nginx and application deployment, with idempotent playbooks
End-to-end cloud deployment	20%	Containerized application running on EC2 and reachable on the public IP, port 80
Validation and technical report	20%	Smoke-test logs, architecture diagrams and the GitHub repository link

9.2 Planned Repository Layout

```
mini-project-4/
  README.md
  terraform/
    main.tf  variables.tf  outputs.tf
  ansible/
    inventory  playbook.yml
    roles/ common  docker  nginx  student_app
```

10. Conclusion

Infrastructure automation turns the last manual part of the delivery chain into code. Terraform describes and creates the AWS network, firewall, server and storage; Ansible then installs Docker and Nginx and starts the Student Record application on that server. Because both are text files stored in Git, the whole environment can be reviewed, repeated and removed on demand, and the same design can be reused across development, staging and production. Together with the earlier units (Git, Jenkins, Docker and Kubernetes), this completes the path from a line of code to a running, reproducible service in the cloud.

Glossary of Key Terms

Term	Meaning
IaC	Infrastructure as Code: managing infrastructure through definition files
Terraform	Declarative tool that provisions cloud resources

Term	Meaning
HCL	HashiCorp Configuration Language used in Terraform files
State file	Record linking declared resources to real cloud objects
Ansible	Agentless tool that configures servers over SSH
Playbook / Role	YAML automation file / reusable group of tasks
Idempotent	Safe to run repeatedly with the same result
VPC / Subnet	Private cloud network / a segment of it
Security Group	Stateful virtual firewall around an instance
Configuration drift	Servers slowly becoming different from each other